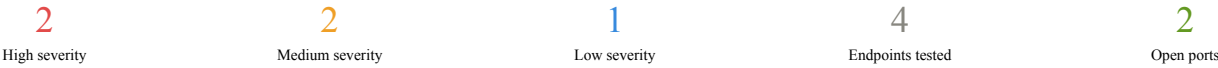


SaaS API Security Risk Analysis Report — interactive dashboard for localhost:3000 assessment dated 21 May 2026

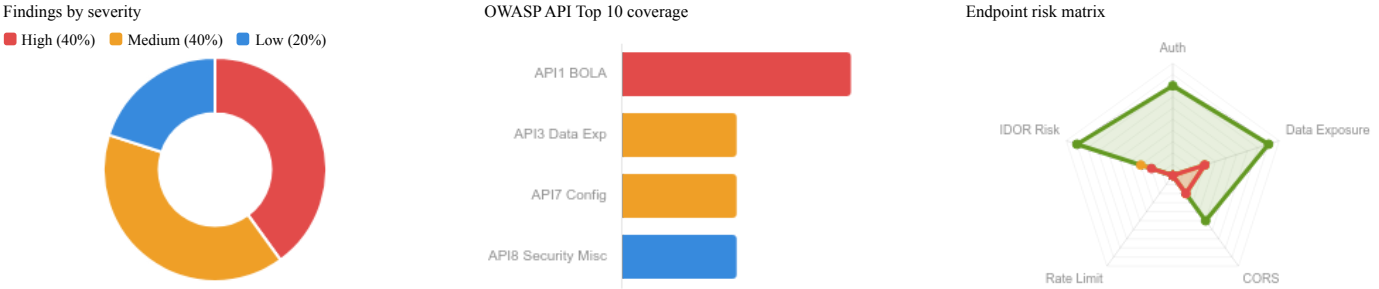
SaaS API Security Risk Analysis

Moderate-High Risk

Target: localhost:3000 · Stack: Node.js / Express / Helmet · Date: 21 May 2026 · Type: Local Read-Only



Risk distribution & OWASP API mapping



Endpoint inventory

GET /	Auth required	Response	Risk	None needed {"status":"SaaS API Running"}	Low — informational only	GET /health	Auth required	Response	Risk	None — publicly exposed {"api":"healthy"}	Low — service state disclosure
GET /api/users	Auth required	Exposes	Risk	None — intentional flaw id, name, email, role, internal_id	High — BOLA + data exposure	GET /api/users/:id	Auth required	IDOR via	Risk	None — direct object ref /api/users/1, /api/users/2	High — BOLA (IDOR)
GET /swagger & GET /docs — probed, not found Both return HTML error page "Cannot GET /swagger" and "Cannot GET /docs" — no API documentation endpoint exposed. Positive finding.											

Detailed findings

FINDING	SEVERITY	OWASP	DETAIL
Unauthenticated /api/users	High	API1:2023	Full user roster (names, emails, roles, internal IDs) returned with zero authentication. Any network-accessible client can enumerate all users.
BOLA / IDOR on /api/users/:id	High	API1:2023	Records accessed by sequential integer ID (/api/users/1, /2). No ownership or session check; any caller can retrieve any user object.
Excessive data exposure	Medium	API3:2023	Responses include role and internal_id (INT-001, INT-002) — fields unnecessary for client rendering; exposes internal system identifiers.
Wildcard CORS policy	Medium	API7:2023	Access-Control-Allow-Origin: * set globally via cors() middleware with no origin restriction — any cross-origin browser request is permitted.
Health endpoint public	Low	API8:2023	/health discloses service operational status without authentication; could aid reconnaissance in a production context.

HTTP response header assessment

HEADER	VALUE / STATUS
Access-Control-Allow-Origin	* (wildcard — overly permissive)
Content-Security-Policy	default-src 'self'; script-src 'self'; object-src 'none' (Helmet default — good)
Strict-Transport-Security	max-age=31536000; includeSubDomains (set but HTTP-only environment)
X-Content-Type-Options	nosniff ✓
X-Frame-Options	SAMEORIGIN ✓

Cross-Origin-Opener-Policy	same-origin ✓
Cross-Origin-Resource-Policy	same-origin ✓
Referrer-Policy	no-referrer ✓
X-DNS-Prefetch-Control	off ✓
X-Permitted-Cross-Domain-Policies	none ✓
X-XSS-Protection	0 (correctly disabled — deprecated header) ✓
ETag	W/"1d-ZboWR4..." — weak ETag fingerprints response body

Network & infrastructure

Nmap scan — localhost (127.0.0.1)	Technology stack
Port 80/tcp — open — http	Runtime: Node.js (Express 4.22.2)
Port 3000/tcp — open — ppp (Node.js API)	Security middleware: Helmet 8.1.0
998 ports closed (reset)	CORS middleware: cors 2.8.6
Scan time: 0.13 seconds	Data store: in-memory array (no DB)
Host: up (0.0000050s latency)	Auth: none implemented
	Rate limiting: none detected

Source code security review — server.js

Positive findings	Vulnerabilities in code
✓ Helmet middleware applied globally	✗ cors() with no options = wildcard
✓ express.json() body parser set	✗ /api/users — no auth middleware
✓ 404 handling on unknown user IDs	✗ /api/users/:id — no auth middleware
✓ No eval(), no exec() usage	✗ All user fields returned (no field filter)
✓ No .env secrets in source	✗ No rate limiting middleware
✓ Port 3000 — no privileged port	✗ No input sanitisation on :id param

Prioritised remediation

- 1
- Implement authentication middleware — add JWT or session-based auth to all /api/* routes. Reject unauthenticated requests with HTTP 401 before any data is accessed.
- 2
- Add authorisation checks on /api/users/:id — verify the requesting user owns the record or has admin role. Return HTTP 403 on ownership mismatch to prevent BOLA/IDOR.
- 3
- Filter response fields — return only id , name , email to clients. Strip role and internal_id from all user-facing responses.
- 4
- Restrict CORS origin — replace cors() with cors({"{"}origin: 'https://yourdomain.com'{"}})) . Wildcard CORS combined with unauthenticated endpoints is a critical misconfiguration.
- 5
- Add rate limiting & protect /health — use express-rate-limit on all routes. Gate /health behind internal network or a bearer token to prevent unauthenticated service reconnaissance.